

Sistemas Digitales

Módulo Sistemas Informáticos — DAM

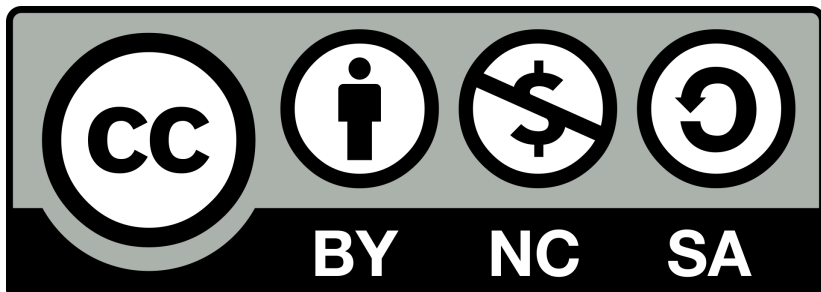
José Gaspar Sánchez García

October 1, 2026

Outline

- 1 Licencia
- 2 Introducción
- 3 Álgebra de Boole
- 4 Puertas lógicas
- 5 Análisis de circuitos digitales
- 6 Síntesis de circuitos digitales
- 7 Formas canónicas
- 8 Mapas de Karnaugh
- 9 Ejemplo práctico 1: 3 variables
- 10 Ejemplo práctico 2: 4 variables
- 11 Ejemplo práctico 3: detector de mayoría (3 variables)
- 12 Logisim: diseño y simulación
- 13 Ejercicios propuestos
- 14 Recursos y referencias
- 15 Coclusión

Este documento se distribuye bajo la licencia Creative Commons Atribución 4.0 Internacional (CC BY-NC-SA 4.0).



- Comprender los fundamentos del álgebra de Boole y su aplicación a circuitos digitales.
- Identificar y usar puertas lógicas básicas y compuestas.
- Analizar y sintetizar circuitos combinacionales.
- Manejar formas canónicas (SdP y PdS) y simplificación con mapas de Karnaugh.
- Introducir Logisim como herramienta de diseño y simulación.

¿Qué es un sistema digital?

- Sistema que opera con señales discretas (normalmente 0 y 1).
- Base de la lógica de computadoras, microcontroladores y circuitos integrados.
- Se modela mediante funciones booleanas y se implementa con puertas lógicas.

- Variables booleanas: toman valores $\{0, 1\}$.
- Operaciones básicas:
 - Suma lógica (OR): $a + b$
 - Producto lógico (AND): $a \cdot b$ o ab
 - Negación (NOT): $\bar{a}$ o a'

Axiomas fundamentales:

- Propiedad Conmutativa:

$$a + b = b + a$$

,

$$a \cdot b = b \cdot a$$

- Propiedad Asociativa:

$$a + (b + c) = (a + b) + c$$

,

$$a \cdot (b \cdot c) = (a \cdot b) \cdot c$$

- Propiedad Distributiva:

$$a \cdot (b + c) = a \cdot b + a \cdot c$$

,

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

Axiomas fundamentales (II):

- Elemento neutro: $a + 0 = a$, $a \cdot 1 = a$.
- Complemento: $a + \bar{a} = 1$, $a \cdot \bar{a} = 0$.
- Idempotencia: $a + a = a$, $a \cdot a = a$.
- Absorción: $a + a \cdot b = a$, $a \cdot (a + b) = a$.
- Dualidad: intercambiar $+$ $\leftrightarrow \cdot$, $0 \leftrightarrow 1$.

- Leyes de De Morgan:

- $\overline{a + b} = \bar{a} \cdot \bar{b}$

- $\overline{a \cdot b} = \bar{a} + \bar{b}$

- NOT (inversor): $\bar{a}$
- AND: $a \cdot b$
- OR: $a + b$
- Representación simbólica y tablas de verdad asociadas.

Puertas básicas (II)

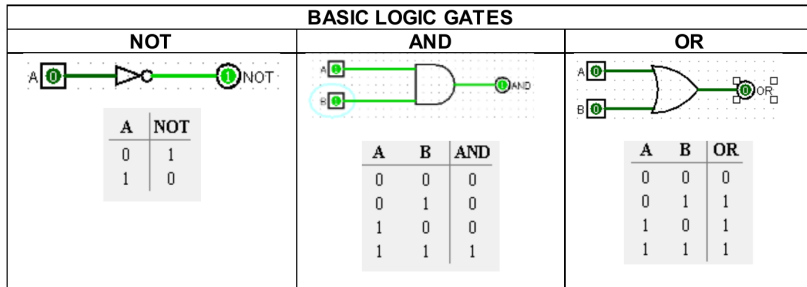


Figure: Puertas lógicas básicas

Puertas compuestas

- NAND: $\overline{a \cdot b}$ (universal)
- NOR: $\overline{a + b}$ (universal)
- XOR: $a \oplus b = a\bar{b} + \bar{a}b$
- XNOR: $\overline{a \oplus b}$
- Todas pueden construirse a partir de NAND o NOR.

Puertas compuestas (II)

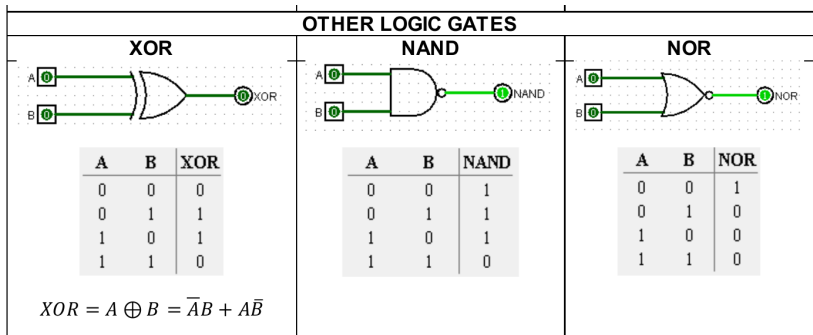


Figure: Puertas lógicas compuestas

Tablas de verdad resumen

a	b	a AND b	a OR b	a XOR b	a NAND b	a NOR b
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

De circuito a función booleana

- Identificar entradas y salidas.
- Seguir el camino de señales a través de puertas.
- Escribir la expresión booleana resultante.
- Obtener la tabla de verdad evaluando para todas las combinaciones.

Ejemplo de análisis

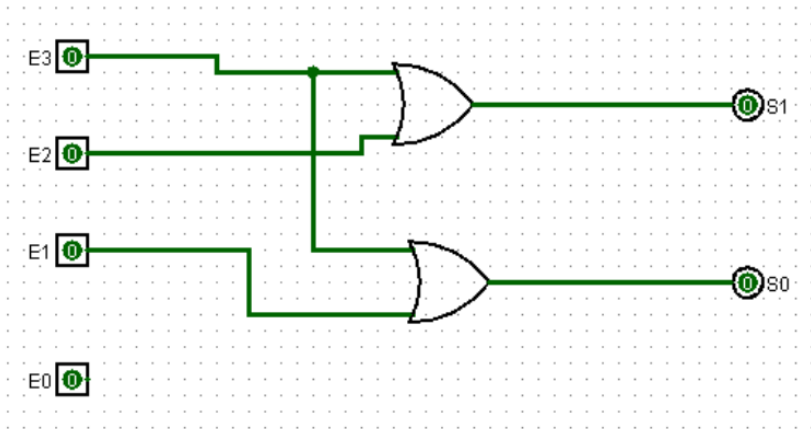


Figure: Analiza el comportamiento del siguiente circuito digital

- Identificación de de entradas y salidas.
 - Entradas: E_3, E_2, E_1, E_0 .
 - Salidas: S_1, S_0 .

Expresiones booleanas resultantes

- $S_1 = E_3 + E_2$
- $S_0 = E_3 + E_1$

Análisis de circuitos digitales (II)

E ₃	E ₂	E ₁	E ₀	S ₁	S ₀
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1	1	1	0
1	0	0	0	1	1
1	0	0	1	1	1
1	0	1	0	1	1
1	0	1	1	1	1
1	1	0	0	1	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	1	1

De especificación a circuito

- Partir de:
 - Tabla de verdad, o
 - Descripción verbal del comportamiento.
- Obtener función booleana (formas canónicas).
- Simplificar (álgebra o Karnaugh).
- Implementar con puertas disponibles.

- Minimizar número de puertas.
- Minimizar niveles de lógica (retardo).
- Considerar tecnologías (TTL, CMOS, FPGA, etc.).
- En DAM: énfasis en comprensión conceptual y simulación.

Minterms y maxterms

- Minterm: producto de todas las variables (directas o negadas) que da 1 en una fila.
- Maxterm: suma de todas las variables que da 0 en una fila.
- Para n variables hay 2^n minterms/maxterms posibles.

Primera forma canónica: Suma de Productos (SdP)

- Se suman los minterms correspondientes a las filas donde $F = 1$.
- Ejemplo para 3 variables A, B, C :
 - Si $F = 1$ en las filas 1, 3, 6, 7:

$$F(A, B, C) = \sum m(1, 3, 6, 7)$$

- Expresión:

$$F = \overline{A}\overline{B}C + \overline{A}BC + AB\overline{C} + ABC$$

Segunda forma canónica: Producto de Sumas (PdS)

- Se multiplican los maxterms correspondientes a las filas donde $F = 0$.
- Ejemplo:

$$F(A, B, C) = \prod M(0, 2, 4, 5)$$

$$F = (A + B + C) \cdot (A + \overline{B} + C) \cdot (\overline{A} + B + C) \cdot (\overline{A} + B + \overline{C})$$

- Representación gráfica de la tabla de verdad.
- Cada celda = una combinación de entradas.
- Celdas adyacentes difieren en una sola variable (código Gray).
- Agrupar 1s (para SdP) o 0s (para PdS) en potencias de 2: 1, 2, 4, 8...

Mapa de 3 variables

- Variables: A (filas), BC (columnas en orden 00, 01, 11, 10).
- Ejemplo de mapa:

$A \setminus BC$	00	01	11	10
0	0	1	1	0
1	0	0	1	1

- Agrupaciones posibles:
 - Grupos de 2. Agrupación 1: 001 y 011. Agrupación 2: 111 y 110.
 - Función simplificada:

$$F = \overline{A}C + AB$$

Mapa de 4 variables

- Variables: AB (filas), CD (columnas), ambas en código Gray.
- Reglas:
 - Los bordes son adyacentes (mapa “envolvente”).
 - Buscar grupos lo más grandes posible.
 - Evitar grupos redundantes si no aportan simplificación.

Pasos para simplificar con Karnaugh

- 1 Construir tabla de verdad.
- 2 Rellenar el mapa con 1s (o 0s).
- 3 Formar grupos de 1s en potencias de 2.
- 4 Extraer términos simplificados.
- 5 Escribir la función mínima.

Especificación del problema

- Función de 3 variables A, B, C que vale 1 cuando:
 - Al menos dos de las tres entradas son 1.
- Objetivo:
 - Obtener tabla de verdad.
 - Expresión canónica (SdP).
 - Simplificación con Karnaugh.
 - Implementación con puertas.

Tabla de verdad

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Filas con $F = 1$: 3, 5, 6, 7 (en numeración desde 0).

- Minterms:

- $m_3 = \overline{A} B C$
- $m_5 = A \overline{B} C$
- $m_6 = A B \overline{C}$
- $m_7 = A B C$

- Función canónica:

$$F(A, B, C) = \sum m(3, 5, 6, 7)$$

$$F = \overline{A} B C + A \overline{B} C + A B \overline{C} + A B C$$

Mapa de Karnaugh (3 variables)

- Orden: filas A , columnas BC en Gray: 00, 01, 11, 10.

$A \setminus BC$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

- 1s en las celdas: $(A=0, BC=11)$, $(A=1, BC=01)$, $(A=1, BC=11)$, $(A=1, BC=10)$.

Agrupaciones y simplificación

- Grupo 1: columna $BC = 11$ (dos celdas: $A=0$ y $A=1$).
 - Variables constantes: $B = 1, C = 1 \rightarrow$ término BC .
- Grupo 2: fila $A = 1$, columnas $BC = 01, 11, 10 \rightarrow$ agrupamos 4 celdas:
 - En realidad, podemos formar un grupo de 4:
 - Filas: $A=1$ (fija)
 - Columnas: 01, 11, 10 y 00 no todas son 1, pero sí 01,11,10.
 - Mejor agrupación:
 - Grupo de 2: $(A=1, BC=01)$ y $(A=1, BC=11) \rightarrow AC$.
 - Grupo de 2: $(A=1, BC=11)$ y $(A=1, BC=10) \rightarrow AB$.
- Función simplificada (una opción mínima):

$$F = AB + AC + BC$$

- Interpretación: “al menos dos entradas a 1”.

- Expresión: $F = AB + AC + BC$
- Circuito:
 - 3 puertas AND de 2 entradas: AB , AC , BC .
 - 1 puerta OR de 3 entradas para sumar los tres términos.
- En Logisim:
 - 3 pines de entrada: A, B, C.
 - 3 AND, 1 OR, 1 pin de salida F.

Especificación del problema

- Función de 4 variables A, B, C, D que vale 1 cuando el número binario $ABCD$ es:
 - Primo y mayor que 2: es decir, 3, 5, 7, 11, 13.
- Valores decimales y binarios:
 - 3 $\rightarrow$ 0011
 - 5 $\rightarrow$ 0101
 - 7 $\rightarrow$ 0111
 - 11 $\rightarrow$ 1011
 - 13 $\rightarrow$ 1101

Tabla de verdad (resumida)

Listamos solo las filas con $F = 1$:

A	B	C	D	Decimal	F
0	0	1	1	3	1
0	1	0	1	5	1
0	1	1	1	7	1
1	0	1	1	11	1
1	1	0	1	13	1
resto de combinaciones		0			

- Minterms: $m_3, m_5, m_7, m_{11}, m_{13}$.

$$F(A, B, C, D) = \sum m(3, 5, 7, 11, 13)$$

$$F = \overline{A} \overline{B} C D \quad (m_3)$$

$$+ \overline{A} B \overline{C} D \quad (m_5)$$

Expresión desarrollada: $\backslash [$

$$+ \overline{A} B C D \quad (m_7) \quad \backslash]$$

$$+ A \overline{B} C D \quad (m_{11})$$

$$+ A B \overline{C} D \quad (m_{13})$$

Mapa de Karnaugh (4 variables)

- Filas: AB en orden Gray: 00, 01, 11, 10.
- Columnas: CD en orden Gray: 00, 01, 11, 10.

Marcamos 1 en las celdas correspondientes a:

- $m_3 \rightarrow AB=00, CD=11$
- $m_5 \rightarrow AB=01, CD=01$
- $m_7 \rightarrow AB=01, CD=11$
- $m_{11} \rightarrow AB=10, CD=11$
- $m_{13} \rightarrow AB=11, CD=01$

Mapa (1 = uno, 0 = cero):

$AB \backslash CD$	00	01	11	10
00	0	0	1	0
01	0	1	1	0
11	0	1	0	0
10	0	0	1	0

Agrupaciones posibles

- Grupo 1: columna $CD=11$, filas $AB=00$ y $01 \rightarrow 2$ celdas.
 - Variables constantes: $C = 1, D = 1$, A y B cambian $\rightarrow$ término $C D$.
- Grupo 2: columna $CD=01$, filas $AB=01$ y $11 \rightarrow 2$ celdas.
 - Variables constantes: B cambia, A cambia, $C = 0, D = 1 \rightarrow$ término $\overline{C} D$.
- Grupo 3: celda restante ($AB=10, CD=11$) ya está cubierta por el grupo 1 ($CD=11$).
 - No hace falta grupo adicional si ya cubrimos todos los 1s.

Revisando:

- 1s cubiertos:
 - $CD=11$: $AB=00,01,10 \rightarrow$ todas cubiertas por grupo $CD=11$.
 - $CD=01$: $AB=01,11 \rightarrow$ cubiertas por grupo $\overline{C} D$.
- Función simplificada:

$$F = C D + \overline{C} D = D (C + \overline{C}) = D$$

- Resultado sorprendente pero correcto: en este conjunto concreto de primos elegidos, la función se reduce a $F = D$.
 - Comprobación rápida: todos los números primos seleccionados ($3,5,7,11,13$) tienen $D = 1$.

Implementación con puertas

- Función simplificada: $F = D$
- Circuito:
 - Simplemente conectar la entrada D a la salida F (sin puertas).
- En Logisim:
 - 4 pines de entrada: A, B, C, D.
 - 1 pin de salida F conectado directamente a D.

- Función de 3 variables A, B, C que vale 1 si la mayoría de las entradas son 1.
- Equivalentemente: “al menos dos entradas a 1” (mismo ejemplo práctico 1, pero con otro nombre).

Tabla de verdad

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- Ya obtenida antes:

$$F = AB + AC + BC$$

- Implementación:
 - 3 AND de 2 entradas.
 - 1 OR de 3 entradas.

¿Qué es Logisim?

- Herramienta gráfica libre para diseñar y simular circuitos digitales.
 - Dibujar circuitos con puertas.
 - Probar con interruptores y LEDs.
 - Visualizar tablas de verdad y cronogramas.

Flujo de trabajo típico

- 1 Crear nuevo circuito.
- 2 Añadir entradas (pines) y salidas.
- 3 Colocar puertas lógicas y conectar cables.
- 4 Usar “poke tool” para cambiar valores de entrada.
- 5 Observar comportamiento y validar con tabla de verdad.

- Diseñar un circuito que implemente:

$$F(A, B, C) = \overline{A} B + A \overline{C}$$

- Pasos:
 - Crear entradas A, B, C.
 - Implementar $\overline{A}$, $\overline{C}$.
 - Construir los productos y la suma.
 - Verificar con todas las combinaciones.

Ejercicio 1: Análisis

- Dado el circuito (imagen anterior):
 - Obtener la función booleana.
 - Construir la tabla de verdad.
 - Simplificar usando álgebra de Boole.

Ejercicio 2: Síntesis con Karnaugh

- Especificación:
 - Función de 3 variables que vale 1 cuando:
 - Al menos dos entradas son 1.
- Tareas:
 - Tabla de verdad.
 - Mapa de Karnaugh.
 - Función simplificada.
 - Implementación con puertas AND, OR, NOT.

Ejercicio 3: Logisim

- Implementar en Logisim:
 - Un comparador de 2 bits (A_1A_0 vs B_1B_0) que indique:
 - $A > B$, $A = B$, $A < B$.
- Entregar:
 - Captura del circuito.
 - Breve memoria explicando el diseño.

- Documentación de Logisim: <https://logisim.sourceforge.net/>

- El álgebra de Boole es el lenguaje matemático de los circuitos digitales.
- Las formas canónicas permiten pasar de tabla de verdad a expresión lógica.
- Los mapas de Karnaugh facilitan la simplificación manual.
- Logisim es una herramienta práctica para validar diseños antes de implementarlos.

- Profundizar en sistemas secuenciales (biestables, contadores, máquinas de estados).
- Introducir VHDL/Verilog como lenguajes de descripción de hardware.
- Relacionar con arquitectura de computadores y organización del procesador.